

Context Loss in AI-Assisted Software Development: Mechanisms, Systemic Impacts, and a Context Engineering Framework

Jehoshua M

Independent Researcher

Abstract

The rapid integration of Large Language Models (LLMs) into software development workflows has catalyzed significant productivity gains in bounded, single-session tasks. However, as development workflows scale to multi-session, real-world project environments, a critical structural limitation emerges: context loss. This paper synthesizes empirical findings from LLM architecture research, human-computer interaction studies, and industry-wide DevOps reports to examine the phenomenon of context loss in AI-assisted software development. We analyze the "lost in the middle" positional degradation mechanism inherent to Transformer-based models and map this technical limitation to systemic developer-facing friction, including the "verification tax," delivery instability, and the hidden cognitive costs of repetitive prompting. To address these gaps, we propose the Context Engineering Stack, a three-layer framework (Static, Dynamic, and Retrieval Context) that formalizes the shift from task-level prompt engineering to systemic context management, complete with layer-interaction logic. Finally, we outline engineering mitigations, a proposed evaluation methodology, and organizational capabilities necessary to sustain long-term Return on Investment (ROI) in AI-assisted development.

Index Terms: Artificial Intelligence, Context Loss, Software Engineering, LLMs, DevOps, Developer Productivity, Context Engineering, Retrieval-Augmented Generation.

I. Introduction

The adoption of AI coding assistants has transitioned from a novelty to a ubiquitous standard in software engineering. Recent industry reports indicate that the vast majority of technology professionals utilize AI tools in their daily workflows [1]. While controlled studies consistently demonstrate speed improvements on well-defined, bounded tasks [2], [3], a growing body of research reveals that these gains do not seamlessly translate to system-level delivery

performance or sustained multi-session project development [1], [4].

The core friction underlying this divergence is "context loss", the failure of an AI system to robustly access, retain, or utilize relevant project information across long input contexts or multiple sessions. Context in AI-assisted development operates on two interconnected planes: the technical context window (the token capacity of the model) and the practical project context (architectural decisions, coding conventions, and institutional knowledge). Current literature reveals a significant gap between nominal context window sizes and the effective, reliable utilization of these windows [5], [6].

This paper investigates the mechanisms of context loss, drawing on foundational LLM behavioral research and comprehensive human-subjects studies. By framing context loss not merely as an LLM architectural quirk, but as a systemic socio-technical challenge, we explore how organizations can leverage platform engineering and a proposed Context Engineering Stack to mitigate the hidden costs of AI-assisted development.

II. Methodology

This research employs a structured literature review and evidence synthesis methodology to investigate context loss in AI-assisted software development. The search strategy targeted four primary academic and industry databases: IEEE Xplore, ACM Digital Library, arXiv, and Google Scholar.

The inclusion criteria comprised peer-reviewed conference papers (e.g., ICSE, CHI), arXiv preprints focusing on LLM architectural limits and developer workflows, and large-scale industry reports (e.g., DORA, Stack Overflow) published between 2020 and 2026. Papers were excluded if they focused solely on model pre-training or lacked empirical evaluation of downstream tasks. Due to the nascent nature of multi-session AI development research, selection bias was mitigated by cross-referencing industry survey data with controlled laboratory experiments, acknowledging that no single methodology currently captures the full spectrum of real-world development workflows. The synthesis identifies key mechanisms, systemic impacts, and emerging mitigations, resulting in a proposed framework for context engineering.

III. Preliminaries and Formal Definitions

To establish a rigorous foundation for analyzing context loss in software engineering, we define the following core constructs:

Definition 1 (Context Loss Event): An instance where an LLM generates an output that violates an explicitly stated project constraint, architectural decision, or previously established session state, due to the positional degradation, eviction, or absence of that information within the model's active attention span.

Definition 2 (Verification Tax): The proportion of developer interaction time spent evaluating, correcting, or re-prompting the AI assistant, rather than generating or accepting code. This represents the hidden cognitive and temporal overhead required to ensure AI output aligns with project reality.

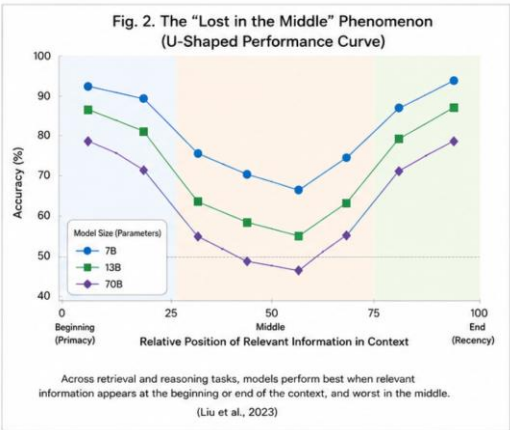
Definition 3 (Context Engineering): The systemic practice of building persistent, project-level context artifacts and retrieval pipelines that shape all LLM interactions, shifting the burden of context retention from the ephemeral conversation window to engineered external systems.

IV. The Mechanism of Context Loss in LLMs

As language models process input sequences, their ability to utilize information is not uniform. The foundational empirical understanding of this limitation is established by the "lost in the middle" phenomenon.

A. Positional Degradation and the U-Shaped Performance Curve

Research by Liu et al. demonstrates that model performance on retrieval and reasoning tasks degrades significantly when relevant information is positioned in the middle of long input contexts [5]. In multi-document question answering and key-value retrieval tasks, performance follows a distinctive U-shaped curve, as illustrated in Fig. 2.



Models exhibit high accuracy when relevant information is placed at the very beginning (primacy bias) or the very end (recency bias) of the input context. Conversely, performance

drops substantially when the model must access and reason over information located in the middle.

This phenomenon persists even in models explicitly designed and marketed for extended-context operation. Empirical evaluations show that extended-context models do not necessarily outperform their standard counterparts at utilizing input contexts that fit within both windows [5]. Furthermore, subsequent research indicates that the effective context length—the length at which performance remains reliable—is typically a fraction of the nominal training context length for most open-source models [6].

B. Architectural and Training Influences

The underlying causes of context loss are linked to both model architecture and training paradigms:

- **Model Architecture:** Encoder-decoder models demonstrate relative robustness to positional changes when evaluated within their training-time sequence length, likely due to bidirectional encoding. However, when evaluated on sequences longer than those seen during training, they too exhibit the U-shaped performance curve [5].
- **Instruction Fine-Tuning:** While instruction fine-tuning improves absolute performance, it does not eliminate the U-shaped curve. Base models and their instruction-tuned counterparts exhibit similar positional biases, indicating that the degradation is rooted in pre-training dynamics rather than fine-tuning protocols [5].
- **Model Scale:** The emergence of primacy bias is linked to model scale. Smaller models (e.g., 7B parameters) are solely recency-biased, while larger models (13B and 70B parameters) begin to exhibit the dual primacy and recency biases characteristic of the U-shaped curve [5].

In software development, this positional degradation implies that architectural constraints or requirements established early in a long conversation history may fall into the "middle" of the context window and be systematically underweighted or ignored by the model.

V. Systemic Impacts on Software Development Workflows

The technical limitations of LLM context utilization manifest as significant systemic frictions in real-world software development, creating hidden costs that standard productivity metrics often fail to capture. The progression from technical context limits to developer friction is depicted in Fig. 3.

engineering systems are not mature enough to absorb this velocity, the organization risks prolonged instability.

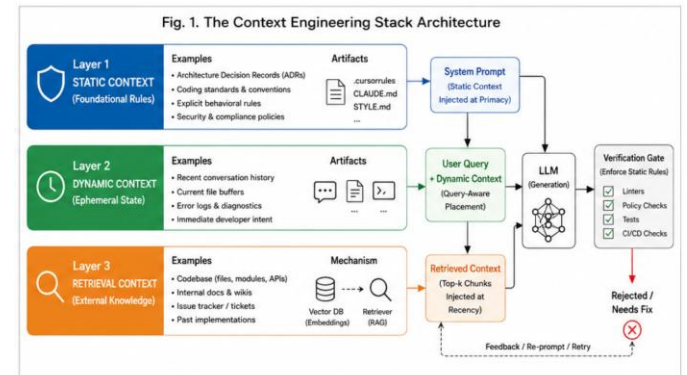
C. The Trust-Adoption Paradox

Context loss is a plausible contributing factor to the erosion of developer trust. Large-scale industry surveys (N≈50,000) reveal a paradox: while AI tool adoption remains near-universal, developer trust in AI output accuracy has been declining [9], [10]. Developers frequently cite "lack of crucial context", such as organizational codebase knowledge and internal architecture, as a primary challenge [9]. This leads to a state where developers continue using tools whose output they actively distrust, implying that the verification overhead is currently treated as an unavoidable cost of doing business. While survey correlation does not equate to strict causation, the consistent attribution of friction to missing project context strongly suggests context loss is a primary driver of this distrust.

VI. Proposed Framework: The Context Engineering Stack

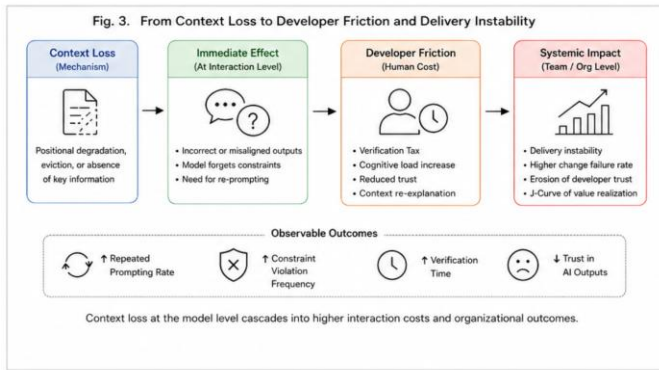
The traditional approach to AI interaction task-level prompt engineering, optimizes individual queries but fails to address the systemic loss of project context across sessions. To mitigate context loss, the industry is shifting toward context engineering [11], [12].

To formalize this shift, we propose the Context Engineering Stack, a three-layer model that categorizes how context is supplied to LLMs, where context gets lost, and how each layer can be mitigated. The architecture of this stack is presented in Fig. 1.



Layer 1: Static Context Layer

This layer comprises persistent, project-wide rules that do not change frequently. It includes architecture decision records (ADRs), coding style conventions, and explicit behavioral prohibitions.



A. The Verification Tax and Hidden Interaction Costs

While AI tools accelerate code generation, they simultaneously introduce a "verification tax"—the cognitive and temporal effort required to review, validate, and correct AI-generated outputs. Because models lose context over long sessions or fail to retain project-specific constraints, they frequently produce outputs that are superficially plausible but architecturally misaligned.

This verification burden constitutes a substantial portion of the AI-assisted development lifecycle. Observational research by Mozannar et al. developed the CUPS (CodeRec User Programming States) taxonomy, revealing that developers spend a significant portion of their AI interaction time on activities other than accepting suggestions, including verifying output correctness, navigating code, re-prompting to correct the model, and debugging AI-introduced errors [7]. Similarly, survey-based research (N=410) by Liang et al. found that developers frequently report difficulty controlling output quality and evaluating the correctness of generated code [8]. The friction does not vanish; it shifts from manual code generation to decision-making and validation.

B. Delivery Instability and the J-Curve of Value Realization

The systemic impact of context loss is observable in team-level delivery metrics. According to large-scale, survey-based industry research (N≈3,000), while AI adoption is associated with improvements in individual throughput, it is concurrently linked to an increase in software delivery instability [1], [4]. AI-accelerated code generation produces larger, less-managed change sets that can overwhelm existing review pipelines and manual gates, potentially contributing to higher change failure rates.

Consequently, organizations may undergo a "J-Curve" of AI value realization [4]. The introduction of AI tools can initially cause a productivity dip driven by the learning curve, workflow friction, and the verification tax. If the underlying

- Mechanism: Developers are increasingly authoring rule files (e.g., .cursorrules, CLAUDE.md) that encode these constraints [11].
- Failure Mode: If not explicitly injected into the system prompt or retrieved via tooling, static constraints are easily forgotten by the model during long sessions.
- Mitigation: Externalizing project context into static files bypasses the "lost in the middle" phenomenon by ensuring constraints are presented at the primacy or recency positions of the context window at query time.

Layer 2: Dynamic Context Layer

This layer represents the ephemeral state of the current development session, including recent conversation history, immediate error logs, and current file buffers.

- Mechanism: Maintained within the model's active context window.
- Failure Mode: Highly susceptible to the U-shaped performance curve [5]. Architectural constraints established twenty messages ago may fall into the "middle" and be underweighted.
- Mitigation: Query-aware contextualization (placing the query before and after the data) has been shown to dramatically improve performance on retrieval tasks by enabling near-perfect utilization of the dynamic window [5].

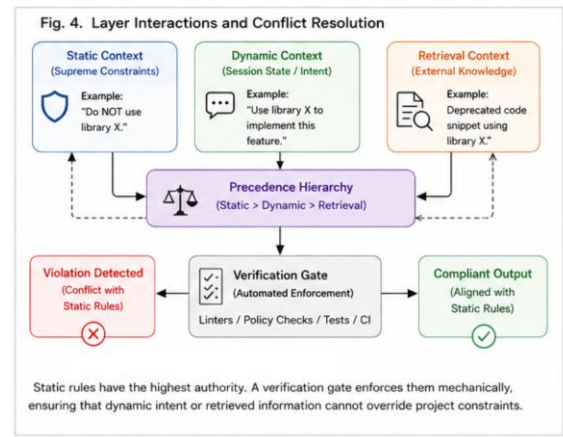
Layer 3: Retrieval Context Layer

This layer handles the dynamic injection of external knowledge at query time, primarily through Retrieval-Augmented Generation (RAG) [13].

- Mechanism: RAG embeds project files into vector stores and retrieves semantically relevant chunks per query. This underlies codebase indexing in tools like Cursor and GitHub Copilot [11].
- Failure Mode: Retrieval misses occur when semantic similarity fails to identify the most architecturally relevant code snippet. Furthermore, RAG impact on multi-session development workflows where context must persist and evolve remains empirically underexplored [11], [13].
- Mitigation: Combining identifier-based and similarity-based RAG across the codebase ensures the model receives necessary project context without exhausting the dynamic context window.

D. Layer Interactions and Conflict Resolution

In practice, these layers do not operate in isolation; they interact, and conflicts emerge. A critical challenge in context engineering is determining precedence when layers contradict, as outlined in Fig. 4.



For example, a developer might issue a dynamic prompt requesting the use of a specific library (Dynamic Layer), while the static rules explicitly prohibit it (Static Layer), and the RAG layer retrieves a deprecated implementation of it (Retrieval Layer).

To resolve this, the stack requires a strict precedence hierarchy and a verification gate:

1. Precedence: Static context must act as the supreme constraint layer (system prompt). Dynamic context and retrieved context should be treated as informational inputs that cannot override static rules.
2. Verification Gate: Because LLMs lack inherent statefulness to reliably arbitrate these conflicts, a post-generation verification gate (e.g., automated linting, CI/CD checks) must enforce static rules mechanically, rather than relying on the model to self-correct.

By mapping AI interactions to these three layers, developers and tooling vendors can systematically diagnose where context loss occurs and apply the appropriate architectural mitigation.

VII. Proposed Evaluation Methodology

While the Context Engineering Stack provides a theoretical model, empirical validation is required. We propose a lightweight, within-subjects alternating-conditions diary study to evaluate the framework's efficacy:

- Conditions: Participants alternate between Condition A (Baseline: standard AI chat without the Context Engineering Stack) and Condition B (Intervention: AI chat utilizing static rule files, RAG indexing, and query-aware dynamic prompting).
- Metrics: Repeated Prompting Rate (number of clarification prompts per session), Constraint Violation Frequency (rate of AI outputs contradicting static rules),

and Verification Time (proportion of session time spent verifying/correcting AI output).

- Goal: Determine if the presence of the Context Engineering Stack statistically significantly reduces the repeated prompting rate and verification tax compared to baseline workflows.

VIII. Engineering Mitigations and Organizational Capabilities

Solving context loss requires more than larger context windows; it demands a holistic engineering and organizational transformation. The DORA AI Capabilities Model provides a data-backed framework of foundational practices that amplify the positive impacts of AI while mitigating instability [1], [4].

A. Quality Internal Platforms and AI-Accessible Data

A quality internal platform serves as the primary connective tissue for AI value [1]. In the agentic era, an Internal Developer Platform (IDP) is no longer just infrastructure; it acts as the context provider and risk mitigator for AI agents. When an organization's internal knowledge is fragmented, AI generates redundant or low-quality code, creating long-term technical debt.

Organizations must cultivate healthy data ecosystems and AI-accessible internal data [1]. By connecting AI tools to high-quality, unified internal data sources, developers provide the company-specific context necessary for the tools to be maximally effective, directly countering the "lack of crucial context" cited by developers [9].

B. Small Batches and Strong Version Control

To manage the delivery instability associated with AI-accelerated code generation, organizations must enforce strong version control practices and working in small batches [1], [4].

Working in small batches ensures that AI-generated code is quickly tested and evaluated, reducing the surface area for context-related errors. Strong version control, particularly proficiency in using rollback features, acts as a "psychological safety net." This safety net allows teams to experiment with AI-generated code and rapidly revert changes when the AI violates an unstated project constraint, thereby absorbing the instability tax [1].

C. User-Centric Focus

Finally, AI adoption must be anchored in a user-centric focus. Without a clear understanding of end-user needs, the increased velocity provided by AI tools may result in rapidly building the wrong features. A user-centric focus ensures that the throughput gains from AI are directed at solving real

problems, guiding the AI-assisted development team toward appropriate goals [1].

IX. Limitations and Future Work

While this synthesis provides a systemic view of context loss, several limitations exist within the current research landscape that constrain generalizability:

1. **Lack of Longitudinal Empirical Studies:** Almost all controlled studies of AI coding assistants examine bounded, single-session tasks [2], [3]. There is a distinct absence of peer-reviewed research measuring context loss and productivity across multi-day, multi-session real-world development workflows where project context accumulates.
2. **Tool-Specific Confounders:** Reproducibility across tools (e.g., GitHub Copilot vs. Cursor vs. Claude Code) is limited. Each tool implements context management (RAG, dynamic windows) differently, making it difficult to isolate whether failures stem from the base LLM or the tool's specific context pipeline.
3. **Self-Report Bias:** Much of the evidence regarding developer trust, the "verification tax," and context-related friction relies on self-reported survey data [8], [9], [10]. While large-scale surveys provide valuable signals, they capture perceptions rather than objective behavioral measurements of context loss events.
4. **Causal Ambiguity:** The link between LLM-level context degradation (the U-shaped curve) and developer-facing outcomes (re-prompting, constraint violations) remains correlational. Future research must operationally define "context loss events" and measure their frequency against specific developer friction points, as proposed in Section VII.

X. Conclusion

Context loss in AI-assisted software development is a multi-dimensional challenge rooted in the architectural constraints of LLMs and amplified by immature engineering systems. The "lost in the middle" phenomenon ensures that nominal expansions in context windows will not, on their own, resolve the degradation of project context over long sessions. This technical limitation is associated with systemic developer friction, manifesting as the verification tax, increased delivery instability, and eroding trust.

To realize the long-term ROI of AI-assisted development, organizations must evolve beyond isolated prompt engineering and tool procurement. The path forward requires systemic context engineering: building robust internal platforms, unifying data ecosystems, and enforcing technical guardrails

like small batches and version control. By treating context as a first-class engineering asset and structuring it through frameworks like the Context Engineering Stack, organizations can transform AI from a localized productivity booster into a sustainable engine for organizational value.

References

- [1] Google Cloud DORA Team, "State of AI-assisted Software Development," DORA Report, 2025. [Online]. Available: <https://dora.dev/dora-report-2025>
- [2] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv preprint arXiv:2302.06590, 2023.
- [3] B. K. Deniz, M. Harrysson, A. Hussin, and S. Srivastava, "Unleashing developer productivity with generative AI," McKinsey & Company, 2023.
- [4] Google Cloud DORA Team, "The ROI of AI-assisted Software Development," DORA Report, 2026. [Online]. Available: <https://dora.dev/ai/roi>
- [5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," arXiv preprint arXiv:2307.03172, 2023.
- [6] C.-P. Hsieh, S. Sun, S. Krause, H. Yu, R. Kambadur, and N. Muennighoff, "RULER: What's the real context window size of your LLM?," arXiv preprint arXiv:2404.06654, 2024.
- [7] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in AI-assisted programming," in Proc. CHI Conf. Human Factors in Computing Systems (CHI), 2024.
- [8] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of AI programming assistants: Successes and challenges," in Proc. 46th IEEE/ACM Int. Conf. Software Engineering (ICSE), 2024, pp. 616–628.
- [9] Stack Overflow, "Stack Overflow Developer Survey 2024," 2024. [Online]. Available: <https://survey.stackoverflow.co/2024>
- [10] Stack Overflow, "Stack Overflow Developer Survey 2025," 2025. [Online]. Available: <https://survey.stackoverflow.co/2025>
- [11] J. M., "Context Loss in AI-Assisted Software Development: A Comprehensive Literature Review and Evidence Synthesis," Internal Non-Peer-Reviewed Technical Report, June 2026.
- [12] Anonymous, "Agent READMEs: An empirical study of context files for agentic coding," arXiv preprint arXiv:2511.12884, 2025.
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.